



OMNIX QUANTUM

OMNIX

Decision Governance Infrastructure

Velos Integration Document | Schema v6.5.4e | T=0 Gateway Edition

For	Naimat Khan — Velos
Prepared by	Harold Nunes — OMNIX QUANTUM LTD
Date	April 09, 2026
Classification	Confidential — Partner Integration Material
Schema Version	v6.5.4e (adds issued_at_ms, ttl_epoch_ms, ttl_ms, signature_format)
Signing Standard	NIST FIPS 204 — ML-DSA-65 (Dilithium-3) / ML-DSA-87 (Dilithium-5)
KEM Standard	NIST FIPS 203 — ML-KEM-768 (Kyber-768)
Endpoint	POST https://velos-gateway.onrender.com/api/v1/intercept

Every OMNIX decision produces a cryptographically signed JSON receipt encoding the complete governance trail. This document defines the exact schema Velos must implement to verify receipts at the T=0 interceptor gateway — TTL enforcement, hash integrity, and PQC signature dispatch.

1. Complete JSON Schema

```
{
  "receipt_id": "OMNIX-A3F7C1D92B0E",
  "timestamp": "2026-04-09T14:23:01.847+00:00",
  "issued_at_ms": 1744208581847, // T=0 reference point (epoch ms)
  "ttl_epoch_ms": 1744208611847, // expiry — REJECT if now_ms > this
  "ttl_ms": 30000, // 30 s default, env: OMNIX_RECEIPT_TTL_MS
  "asset": "BTC/USDT",
  "decision": "BLOCK", // BLOCK | ALLOW | PASS | UNKNOWN
  "veto_chain": ["AML: PASS ...", "SHARIA: BLOCK — gharar=0.81 ..."],
  "policy_version": "6.5.4e",
  "engine_version": "6.5.4e",
  "prev_hash": "a9f3c1d4...", // SHA-256 of previous receipt (chain)
  "signing_provider": "dilithium3", // bound into hash — anti-downgrade
  "content_hash": "3f8e7a2c...", // SHA-256 of canonical payload
  "signature": "BASE64==...", // encoding → see signature_format
  "signature_algorithm": "Dilithium-3 (ML-DSA-65)",
  "signature_format": "base64_pqc", // base64_pqc | hex_sha256_fallback | NONE
  "public_key": "BASE64==...", // null in fallback mode
  // Conditional (present only if gate evaluated):
  "sharia_compliance": {...}, "aml_compliance": {...}, "fraud_compliance": {...},
  "jurisdiction_compliance": {...}, "context_admission": {...}, "avm_result": {...}
}
```

2. Field Reference

Field	Type	Description
receipt_id	string	Unique ID. Format: OMNIX-{12 hex uppercase}
timestamp	string	ISO-8601 UTC generation time
issued_at_ms	integer	Unix epoch ms — canonical T=0 reference point
ttl_epoch_ms	integer	Expiry epoch ms. Velos MUST reject if now_ms > ttl_epoch_ms
ttl_ms	integer	TTL in ms. Default 30000 (30 s). Configurable via OMNIX_RECEIPT_TTL_MS
asset	string	Asset symbol — e.g. BTC/USDT, ETH/USD
decision	string	BLOCK ALLOW PASS UNKNOWN. BLOCK = OMNIX vetoed, Velos must not proceed
veto_chain	array	Ordered gate verdicts. Format: 'GATE: RESULT — detail' (max 80 chars each)
prev_hash	string	SHA-256 of prior receipt — tamper-evident chain. Empty string for first receipt
signing_provider	string	Algorithm ID bound into signed payload (anti-downgrade). Values: dilithium3 dilithium5 sha256
content_hash	string	SHA-256 hex of canonical JSON (sort_keys=True, ensure_ascii=True). This is what gets signed
signature_format	string	Critical for Velos. Disambiguates signature encoding. See Section 3
signature	str null	Detached signature over content_hash. Encoding depends on signature_format
public_key	str null	Base64 public key for PQC verification. null when signature_format=hex_sha256_fallback

3. signature_format — Dispatch Table

signature_format	Signature encoding	How Velos verifies
base64_pqc	Base64-encoded raw PQC bytes	base64.decode → Dilithium.verify(sig, content_hash, public_key)
hex_sha256_fallback	Hex of SHA-256(content_hash). Symmetric. public_key=null	sha256(content_hash.encode()).hexdigest() == signature
NONE	Signing failed — no integrity guarantee	REJECT unconditionally

4. Verification Algorithm (drop-in Python | pip install pypqc)

```
import json, hashlib, base64, time
def verify_omnix_receipt(r):
    # 1 – TTL
    if r.get('ttl_epoch_ms') and int(time.time()*1000) > r['ttl_epoch_ms']:
        return {'valid': False, 'reason': 'RECEIPT_EXPIRED'}
    # 2 – Rebuild signed payload
    M = ['receipt_id', 'timestamp', 'issued_at_ms', 'ttl_epoch_ms', 'ttl_ms',
        'asset', 'decision', 'veto_chain', 'policy_version', 'engine_version',
        'prev_hash', 'signing_provider']
    C = ['sharia_compliance', 'aml_compliance', 'fraud_compliance',
        'jurisdiction_compliance', 'context_admission', 'veto_type', 'avm_result']
    p = {k: r[k] for k in M if k in r}
    for k in C:
        if k in r: p[k] = r[k]
    # 3 – Verify content_hash
    h = hashlib.sha256(json.dumps(p, sort_keys=True, ensure_ascii=True).encode()).hexdigest()
    if h != r.get('content_hash'): return {'valid': False, 'reason': 'HASH_MISMATCH'}
    # 4 – Verify signature
    fmt, sig = r.get('signature_format'), r.get('signature'), r['content_hash'].encode()
    if fmt == 'base64_pqc':
        prov = r.get('signing_provider', 'dilithium3')
        try:
            if prov == 'dilithium3':
                from pqc.sign import dilithium3
                dilithium3.verify(base64.b64decode(sig), msg, base64.b64decode(r['public_key']))
            elif prov == 'dilithium5':
                from pqc.sign import dilithium5
                dilithium5.verify(base64.b64decode(sig), msg, base64.b64decode(r['public_key']))
        except: return {'valid': False, 'reason': 'SIGNATURE_INVALID'}
    elif fmt == 'hex_sha256_fallback':
        if hashlib.sha256(r['content_hash'].encode()).hexdigest() != sig:
            return {'valid': False, 'reason': 'FALLBACK_HASH_MISMATCH'}
        else: return {'valid': False, 'reason': f'UNKNOWN_FORMAT:{fmt}'}
    return {'valid': True}
```

Tier	signing_provider	Algorithm	Standard	Security	PK / Sig
Enterprise (default)	dilithium3	Dilithium-3 (ML-DSA-65)	FIPS 204	~192-bit	1952 B / ~3309 B
High-Assurance	dilithium5	Dilithium-5 (ML-DSA-87)	FIPS 204	~256-bit	2592 B / ~4627 B
No PQC fallback	sha256	SHA-256 symmetric	—	N/A	N/A / 64 hex

5. content_hash — Fields Included / Excluded

IN HASH (mandatory): asset, decision, engine_version, issued_at_ms, policy_version, prev_hash, receipt_id, signing_provider, timestamp, ttl_epoch_ms, ttl_ms, veto_chain
IN HASH (if present): sharia_compliance, aml_compliance, fraud_compliance, jurisdiction_compliance, context_admission, veto_type, avm_result

NOT IN HASH (detached signature pattern): content_hash (itself), signature, signature_algorithm, signature_format, public_key

6. Integration Checklist

#	Check	Rule
1	now_ms > ttl_epoch_ms?	Reject with RECEIPT_EXPIRED
2	Read signature_format	Never hardcode — dispatch on this field
3	Read signing_provider	Determines which PQC algorithm to use
4	pip install pypqc	Required on Velos for PQC verification

5	Recompute content_hash	Exclude: signature, public_key, sig_format, sig_algorithm, content_hash itself
6	decision == BLOCK?	Do not proceed — OMNIX has vetoed this action
7	signature_format == NONE?	Reject unconditionally — no integrity guarantee

Harold Nunes — OMNIX QUANTUM LTD | omnixquantum.com